เอกสารยื่นแจ้งข้อมูลลิขสิทธิ์ (แบบ ลข.01) • กรมทรัพย์สินทางปัญญา

มาตรฐานโครงสร้างไฟล์และชุดโปรแกรมคอมพิวเตอร์ PyGrassReal 4D Dynamic Splat (.py4s)

PyGrassReal 4D Dynamic Splat Single-Container Asset Standard & Codec Specification

ประเภทงานลิขสิทธิ์:	วรรณกรรม (โปรแกรมคอมพิวเตอร์)
ชื่อผู้สร้างสรรค์:	นายปรัชญา บัวผัด (Mr. Prachaya Buapad)
เจ้าของลิขสิทธิ์:	นายปรัชญา บัวผัด / PyGrassReal AI
สัญชาติ / ประเทศ:	ไทย (สร้างสรรค์ในประเทศไทย)
ปีที่สร้างสรรค์แล้วเสร็จ:	พ.ศ. 2569 (ค.ศ. 2026)
แพลตฟอร์มและเว็บไซต์:	https://pygrassreal.ai
คลังโค้ดต้นฉบับ:	github.com/PYGRASSREAL/pygrassreal-py4s-format

สงวนลิขสิทธิ์ พ.ศ. 2569 โดย นายปรัชญา บัวผัด และ PyGrassReal AI (All Rights Reserved)
เอกสารชุดนี้จัดทำขึ้นเพื่อใช้ประกอบการยื่นแจ้งข้อมูลลิขสิทธิ์ต่อกรมทรัพย์สินทางปัญญา กระทรวงพาณิชย์

1. สาระสำคัญของผลงานและแรงบันดาลใจในการสร้างสรรค์

บทสรุปนวัตกรรม (Executive Summary)
ผลงานชิ้นนี้คือการคิดค้น ออกแบบ และพัฒนาสถาปัตยกรรมโครงสร้างไฟล์ไบนารีและชุดโปรแกรมคอมพิวเตอร์ตัวแปลงรหัส (Codec) ในชื่อ **PyGrassReal 4D Dynamic Splat (.py4s)** ซึ่งเป็นมาตรฐานไฟล์บรรจุเดี่ยว (Single-Container) สำหรับเทคโนโลยี **4D Gaussian Splatting (4DGS)** ที่บูรณาการข้อมูลการแสดงผลภาพเคลื่อนไหวเสมือนจริง 4 มิติ เข้ากับการเปลี่ยนรูปทางเวลา (Temporal Motion) และเรขาคณิตการชนทางฟิสิกส์ที่เคลื่อนที่ตามร่างกายแบบ 1:1 รวมอยู่ภายในไฟล์เดียวกัน

1.1 ปัญหาทางเทคโนโลยีในระดับสากล (The 4D Spatial Challenge)

ในอุตสาหกรรมคอมพิวเตอร์กราฟิกส์ขั้นสูง การขยายตัวของเทคโนโลยี 3D Gaussian Splatting เข้าสู่มิติเวลา (Temporal Dimension) เพื่อสร้างโมเดล 4D Dynamic Splatting จากวิดีโอ ประสบปัญหาข้อจำกัดทางวิศวกรรมที่ร้ายแรง 3 ประการ:

- **ปัญหาการกระจัดกระจายของไฟล์ (File Fragmentation):** การเก็บข้อมูล 4DGS ทั่วไปมักใช้วิธีจัดเก็บกลุ่มจุดต่อ 1 เฟรมเป็น 1 ไฟล์ ทำให้คลิปสั้นเพียง 3 วินาที (90 เฟรม) ต้องใช้พื้นที่มากกว่า 1.35 GB และต้องอ่านไฟล์หลายร้อยไฟล์ขึ้นหน่วยความจำ
- **ปัญหาภาพเคลื่อนไหวไร้การตอบสนองทางกายภาพ (Ghost Dynamic Assets):** โมเดล 4DGS ทั่วไปมีเพียงข้อมูลแสง (Radiance) เท่านั้น ไม่มีโครงข่ายการชนทางฟิสิกส์ ทำให้เมื่อนำเข้าสู่เกมเอนจิ้น ตัวละครอื่นหรือกล้องจะเดินทะลุผ่านตัวละครที่กำลังเคลื่อนไหวได้ตลอดเวลา
- **ความคลาดเคลื่อนของการเคลื่อนที่และฟิสิกส์ (Desynchronized Physics):** การทำโครงกระดูก (Rigging) แยกต่างหากเพื่อพยายามให้ตาข่ายชนขยับตามจุดพอยต์คลาวด์ มักเกิดอาการเหลื่อมล้ำทางมิติ สเกลเพี้ยน และกระตุกอย่างรุนแรง

1.2 นวัตกรรมและการสร้างสรรค์ของ PyGrassReal (.py4s)

ผู้สร้างสรรค์ได้ออกแบบและพัฒนาระบบไฟล์ **`.py4s`** ขึ้นเพื่อแก้ไขข้อจำกัดดังกล่าวโดยสิ้นเชิง ด้วยแนวคิด *Single-Container 4D Dynamic Architecture* ซึ่งรวมองค์ประกอบสำคัญ 5 ชั้น (Layers) เข้าสู่โครงสร้างไบนารีระดับลึก:

1. **Visual Layer (Compressed 4DGS Base Payload):** ข้อมูลการกระจายตัวของ 3D Gaussians ในท่าตั้งต้นที่ผ่านการบีบอัดความหนาแน่นสูง ลดขนาดลงได้ถึง 85% โดยคงคุณภาพจุด Gaussians ครบ 100%
2. **Deformation Layer (Temporal Motion Field):** ชุดข้อมูลน้ำหนักและเวกเตอร์การเคลื่อนไหวตามเวลา สำหรับทำนายการกระจัดของตำแหน่ง (Position), การหมุน (Rotation) และการปรับขนาด (Scale) ของแต่ละจุดอย่างต่อเนื่อง
3. **Physics Layer (Deformable Collider):** ตาข่ายเรขาคณิตการชน (Collision Mesh Hitbox) ที่ถูกคำนวณและปรับเปลี่ยนรูปทรงตามการเคลื่อนไหวของกลุ่มจุดแบบเรียลไทม์ 1:1
4. **Metadata Layer (Spatiotemporal Descriptor):** ข้อมูลกำกับพารามิเตอร์เวลา อัตราเฟรม (FPS) ระยะเวลารวม การวนลูป และขอบเขตแกนพิกัด
5. **Visual Preview Layer (Optional Preview):** ภาพหรือวิดีโอเคลื่อนไหวตัวอย่างแบบฝังในตัว สำหรับการแสดงผลพรีวิวทันทีโดยไม่ต้องโหลดไฟล์ทั้งหมด

2. สถาปัตยกรรมโครงสร้างไฟล์ไบนารี (Binary Architecture)

2.1 ข้อมูลคุณลักษณะพื้นฐานของไฟล์

- **ชื่อฟอร์แมต:** PyGrassReal 4D Dynamic Splat Single Container File Format (`.py4s`)
- **รหัสระบุตัวตน (Magic String):** `"PY4S"` (รหัสฐานสิบหก `0x50 0x59 0x34 0x53`)
- **การจัดเรียงไบต์ (Endianness):** Little-Endian (LE) | **ขนาดส่วนหัว (Header Size):** 128 Bytes คงที่
- **เอนจิ้นเป้าหมาย:** Unreal Engine 5, Unity (URP/HDRP), Three.js, Python PyTorch

2.2 ตารางแจกแจงโครงสร้างส่วนหัว (128-Byte Binary Header Layout)

ออฟเซ็ต	ชนิดข้อมูล	ชื่อฟิลด์	คำอธิบายหน้าที่
`0x00 - 0x03`	`char[4]`	`magic`	รหัสระบุชนิดไฟล์ ต้องเป็น `"PY4S"` เสมอ
`0x04 - 0x05`	`uint16`	`version_major`	เลขเวอร์ชันหลัก (ค่าเริ่มต้น: 1)
`0x06 - 0x07`	`uint16`	`version_minor`	เลขเวอร์ชันย่อย (ค่าเริ่มต้น: 0)
`0x08 - 0x0B`	`uint32`	`section_count`	จำนวนส่วนข้อมูลที่บรรจุในไฟล์ (ปกติ: 4 หรือ 5)
`0x0C - 0x0F`	`uint32`	`flags`	บิตแฟล็กคุณลักษณะ (บิต 0: บีบอัดสูง, บิต 1: มี Deformation, บิต 2: มี Collider, บิต 4: มี Preview)
`0x10 - 0x17`	`uint64`	`spz_offset`	ตำแหน่งไบต์เริ่มต้นของ Section 1 (Visual Layer 4DGS Base)
`0x18 - 0x1F`	`uint64`	`spz_size`	ขนาดความยาวไบต์ของ Section 1 (Visual Layer)
`0x20 - 0x27`	`uint64`	`deformation_offset`	ตำแหน่งไบต์เริ่มต้นของ Section 2 (Deformation Layer ข้อมูลการเคลื่อนไหว)
`0x28 - 0x2F`	`uint64`	`deformation_size`	ขนาดความยาวไบต์ของ Section 2 (Deformation Layer)
`0x30 - 0x37`	`uint64`	`collider_offset`	ตำแหน่งไบต์เริ่มต้นของ Section 3 (Physics Layer โครงตาข่ายการชน)
`0x38 - 0x3F`	`uint64`	`collider_size`	ขนาดความยาวไบต์ของ Section 3 (Physics Layer)
`0x40 - 0x47`	`uint64`	`metadata_offset`	ตำแหน่งไบต์เริ่มต้นของ Section 4 (Metadata Layer พารามิเตอร์เวลา)
`0x48 - 0x4F`	`uint64`	`metadata_size`	ขนาดความยาวไบต์ของ Section 4 (Metadata Layer)
`0x50 - 0x57`	`uint64`	`thumbnail_offset`	ตำแหน่งไบต์เริ่มต้นของ Section 5 (Visual Preview Layer ภาพ/วิดีโอตัวอย่าง)
`0x58 - 0x5F`	`uint64`	`thumbnail_size`	ขนาดความยาวไบต์ของ Section 5 (Visual Preview Layer, 0 หากไม่มี)
`0x60 - 0x7F`	`uint8[32]`	`reserved`	พื้นที่สงวนสำหรับส่วนขยายในอนาคต (zero padding 32 ไบต์)

3. ซอร์สโค้ดโปรแกรมคอมพิวเตอร์ต้นฉบับ (Web & TypeScript Codec)

ชุดโปรแกรมคอมพิวเตอร์ตัวอ่าน เขียน และถอดรหัส (Codec Implementation) สำหรับเว็บเบราว์เซอร์ Three.js, WebGL และ WebGPU:

```
// PyGrassReal AI - .py4s Web & TypeScript Codec
// Copyright (c) 2026 PyGrassReal AI (Mr. Prachaya Buapad). All Rights Reserved.
export const PY4S_HEADER_MAGIC = 'PY4S';
export const PY4S_HEADER_SIZE = 128;
export const PY4S_VERSION_MAJOR = 1;
export const PY4S_VERSION_MINOR = 0;

export interface Py4sContainerData {
  spzData: Uint8Array;
  deformationData?: Uint8Array;
  colliderData?: Uint8Array;
  metadata: Record<string, unknown>;
  thumbnailData?: Uint8Array;
}

export class Py4sCodec {
  public static unpack(buffer: ArrayBufferLike | ArrayBufferView): Py4sContainerData {
    const bytes = buffer instanceof Uint8Array
      ? buffer
      : ('buffer' in buffer
          ? new Uint8Array(buffer.buffer, buffer.byteOffset, buffer.byteLength)
          : new Uint8Array(buffer as ArrayBuffer));
    if (bytes.byteLength < PY4S_HEADER_SIZE) throw new Error("Invalid PY4S buffer: size < 128");

    const view = new DataView(bytes.buffer as ArrayBuffer, bytes.byteOffset, bytes.byteLength);
    if (bytes[0] !== 0x50 || bytes[1] !== 0x59 || bytes[2] !== 0x34 || bytes[3] !== 0x53) {
      throw new Error("Not a valid PY4S container (header magic mismatch)");
    }

    const spzOffset = Number(view.getBigUint64(16, true));
    const spzSize = Number(view.getBigUint64(24, true));
    const deformOffset = Number(view.getBigUint64(32, true));
    const deformSize = Number(view.getBigUint64(40, true));
    const colliderOffset = Number(view.getBigUint64(48, true));
    const colliderSize = Number(view.getBigUint64(56, true));
    const metaOffset = Number(view.getBigUint64(64, true));
    const metaSize = Number(view.getBigUint64(72, true));
    const thumbOffset = Number(view.getBigUint64(80, true));
    const thumbSize = Number(view.getBigUint64(88, true));

    const spzData = bytes.slice(spzOffset, spzOffset + spzSize);
    const deformationData = deformSize > 0 ? bytes.slice(deformOffset, deformOffset + deformSize) : undefined;
    const colliderData = colliderSize > 0 ? bytes.slice(colliderOffset, colliderOffset + colliderSize) : undefined;
    const thumbnailData = thumbSize > 0 ? bytes.slice(thumbOffset, thumbOffset + thumbSize) : undefined;
    const metaJson = new TextDecoder('utf-8').decode(bytes.slice(metaOffset, metaOffset + metaSize));

    return { spzData, deformationData, colliderData, metadata: JSON.parse(metaJson), thumbnailData };
  }
}
```

4. ซอร์สโค้ดโปรแกรมคอมพิวเตอร์ต้นฉบับ (Python SDK & Engine Codec)

ชุดโปรแกรมคอมพิวเตอร์ตัวอ่านและเขียนไฟล์ไบนารีระดับลึกสำหรับ AI Pipeline และการเชื่อมต่อไปยังเกมเอนจิ้น:

```
# PyGrassReal AI - .py4s 4D Dynamic Splat Single-Container Codec (Python SDK)
# Copyright (c) 2026 PyGrassReal AI (Mr. Prachaya Buapad). All Rights Reserved.
import struct, json
from dataclasses import dataclass
from typing import Optional, Dict, Any

HEADER_MAGIC = b"PY4S"
HEADER_FORMAT = "<4sHHIIQQQQQQQQQ32s"
HEADER_SIZE = struct.calcsize(HEADER_FORMAT) # 128 Bytes

class Py4sCodec:
    @staticmethod
    def unpack(data: bytes):
        if len(data) < HEADER_SIZE:
            raise ValueError("Data smaller than PY4S header size (128 bytes)")
        header = struct.unpack(HEADER_FORMAT, data[:HEADER_SIZE])
        if header[0] != HEADER_MAGIC:
            raise ValueError(f"Magic mismatch: expected {HEADER_MAGIC}, got {header[0]}")

        spz_off, spz_sz = header[5], header[6]
        def_off, def_sz = header[7], header[8]
        col_off, col_sz = header[9], header[10]
        meta_off, meta_sz = header[11], header[12]
        thumb_off, thumb_sz = header[13], header[14]

        return {
            "spz_data": data[spz_off : spz_off + spz_sz],
            "deformation_data": data[def_off : def_off + def_sz] if def_sz > 0 else None,
            "collider_data": data[col_off : col_off + col_sz] if col_sz > 0 else None,
            "metadata": json.loads(data[meta_off : meta_off + meta_sz].decode("utf-8")),
            "thumbnail_data": data[thumb_off : thumb_off + thumb_sz] if thumb_sz > 0 else None
        }

    @staticmethod
    def pack(spz: bytes, deformation: bytes, collider: bytes, metadata: dict, thumbnail: bytes = b"") -> bytes:
        meta_bytes = json.dumps(metadata).encode("utf-8")
        current_offset = HEADER_SIZE
        # Calculate dynamic offsets for all 5 payload sections
        spz_off, spz_sz = current_offset, len(spz); current_offset += spz_sz
        def_off, def_sz = current_offset, len(deformation); current_offset += def_sz
        col_off, col_sz = current_offset, len(collider); current_offset += col_sz
        meta_off, meta_sz = current_offset, len(meta_bytes); current_offset += meta_sz
        thumb_off, thumb_sz = (current_offset, len(thumbnail)) if thumbnail else (0, 0)

        header = struct.pack(HEADER_FORMAT, HEADER_MAGIC, 1, 0, 5, 0x1 | 0x2 | 0x4,
                             spz_off, spz_sz, def_off, def_sz, col_off, col_sz,
                             meta_off, meta_sz, thumb_off, thumb_sz, b"\x00" * 32)
        return header + spz + deformation + collider + meta_bytes + (thumbnail or b"")
```

5. หลักฐานการสร้างสรรค์และคำรับรองของผู้สร้างสรรค์

5.1 หลักฐานการสร้างสรรค์และการพัฒนา (Proof of Creation)

ผลงานมาตรฐานโครงสร้างไฟล์และชุดโปรแกรมคอมพิวเตอร์ **PyGrassReal 4D Dynamic Splat (.py4s)** ได้รับการพัฒนาขึ้นโดย **นายปรัชญา บัวผัด** ภายใต้โครงการวิจัยและพัฒนา PyGrassReal AI โดยมีหลักฐานการบันทึกเชิงดิจิทัลที่สามารถตรวจสอบย้อนหลังได้ดังนี้:

- **แพลตฟอร์มเผยแพร่ทางการ:** https://pygrassreal.ai
- **คลังสเปกและโค้ดต้นฉบับระดับสากล:** https://github.com/PYGRASSREAL/pygrassreal-py4s-format
- **บันทึก Cryptographic SHA Commit History:** ตรวจสอบได้บนระบบ Git Version Control ซึ่งบันทึกวันเวลาและลายมือชื่อดิจิทัลไว้อย่างถาวร

5.2 คำรับรองความถูกต้องของข้อมูล

ข้าพเจ้า **นายปรัชญา บัวผัด** ขอรับรองว่าข้อมูลรายละเอียดและชุดโปรแกรมคอมพิวเตอร์ที่ปรากฏในเอกสารฉบับนี้ เป็นผลงานที่ข้าพเจ้าได้คิดค้น ออกแบบ และสร้างสรรค์ขึ้นด้วยตนเองโดยสุจริต มิได้คัดลอก ลอกเลียน หรือละเมิดลิขสิทธิ์ของผู้ใดทั้งสิ้น ข้อมูลทั้งหมดที่ระบุไว้ในคำขอแจ้งข้อมูลลิขสิทธิ์ฉบับนี้เป็นความจริงทุกประการ

ลงชื่อ..... ผู้สร้างสรรค์ / ผู้แจ้งข้อมูล

(นายปรัชญา บัวผัด)

ผู้ก่อตั้งและหัวหน้าวิศวกรระบบ PyGrassReal AI

วันที่ ..... เดือน ..... พ.ศ. 2569